

Module 7: Variant Calling and Annotation (Part 1)

Types of sequencing methods

Feature	Whole Genome Sequencing (WGS)	Whole Exome Sequencing (WES)	Targeted Gene Sequencing (TGS)
Scope	Entire genome (coding and non-coding regions)	Exonic regions (~1–2% of the genome)	Deep sequencing of a predefined panel of genes
Advantages	- Most comprehensive view of genetic variation - Detects rare variants in regulatory/intergenic regions	- More cost-effective than WGS - Focuses on protein-coding changes with direct functional impact	- Highly cost-effective for focused questions - Very high coverage of genes
Disadvantages	- High cost and large data volume - Requires substantial computational resources	- Misses non-coding regions - Coverage can be uneven across exons	- Limited to the chosen target region - Cannot detect variants outside the panel

Variant calling pipeline

Quality Control

- Reduces false positives by filtering out low-quality bases, adapters, and contaminants.
1. **FastQC** : Generates summary metrics (per-base quality, GC content, adapter content, sequence duplication).
 - Diagnose common issues (e.g., low average quality at read ends, abnormal GC distribution). **Do we need trimming or other preprocessing?**
 2. **fastp**: Performs both QC and automated read trimming/correction in a single step.

Removes adapters, filters out low-quality reads, corrects mismatches in overlapping paired-end reads.

Read Alignment

- After pre-processing (trimming, quality filtering), reads need to be aligned to a reference genome.
- Aligning reads maps each sequence back to its most likely location in the genome.
- Common Tools:
 - BWA-MEM: Well-suited for short to moderate-length reads; widely used in variant-calling workflows.
 - Bowtie2: Efficient for short reads



Read Mapping: BWA-MEM

Feature	Details
Algorithm	Based on BWT/FM index (a seed-and-extend strategy) specialized for both short and moderate-length reads.
Use Case	Commonly used in DNA variant-calling pipelines (WGS/WES) and can handle somewhat longer reads (up to a few kb).
Pros	- Good balance of speed and accuracy - Handles indels and moderate read lengths effectively
Cons	- Computational cost can be higher than Bowtie2 for certain datasets - Minor speed trade-off for improved sensitivity

Read Mapping: Bowtie2

Feature	Details
Algorithm	Uses a Burrows-Wheeler Transform (BWT) and FM index with a seed-and-extend approach optimized for short reads.
Typical Use Case	Primarily used for aligning short Illumina reads in DNA or RNA datasets.
Pros	- Fast and memory-efficient for short reads - Clear documentation and stable performance
Cons	- Slightly less sensitive for reads with large insertions/deletions - Can be less accurate for very long reads

SAM/BAM Output Files

- SAM: Human-readable text file containing alignment information (e.g., read name, position, CIGAR string).
- BAM: Binary-encoded version of SAM; more compact and faster to process.

What's Inside?

- Positions: Chromosome and start coordinate where the read aligns.
- Flags: Indicate paired-end status, mapped/unmapped, strand orientation, etc.
- **Mapping Quality:** An integer (MAPQ) estimating the confidence in the alignment placement

Mapping Quality

- A numeric score assigned to each read alignment in a SAM/BAM file that reflects the aligner's confidence in that read's placement on the reference genome.
- MAPQ often ranges from 0 to 60 in many aligners (though some can exceed 60).
- **Higher** values indicate **greater** confidence that the read is aligned correctly and uniquely.

Why do we care?

- Reads with low MAPQ are more likely to be mapped to the wrong location or align equally well to multiple regions.
- Many pipelines filter out alignments below a certain MAPQ threshold (e.g., 20 or 30) to improve confidence in variant calling or downstream analyses.

Post Alignment QC Metrics

1. Mapping Rate
2. Coverage Depth
3. Duplication Levels
4. Insert Size Distribution (Paired-End)

Mapping Rate

- Percentage of reads successfully aligned to the reference.

Why do we care?

- Low mapping rate may indicate contamination, poor-quality reads, or reference-genome mismatch affecting variant-calling accuracy.

Tool : `samtools flagstat`

- Provides quick statistics on the number of mapped reads, properly paired reads, duplicates, etc.
- **What to look for:** A high proportion of properly paired reads (for paired-end) and minimal unmapped reads.

Coverage Depth

- Average number of reads covering each base or region.
- Critical for identifying underrepresented areas that may yield false negatives or low confidence calls.

Why do we care?

- We want regions of interest (e.g., exons, target panels) to have sufficient reads for variant detection.
- “Coverage gaps” might call for re-sequencing or caution in interpretation.

Tool: `mosdepth`

- Calculates coverage depth across genome or targeted regions.
- **What to look for:** Low-coverage or uncovered loci

Read Duplication

- Fraction of reads that appear to be duplicates (often from PCR amplification).
- High duplication diminishes effective coverage and can inflate variant confidence.

Why do we care?

- Duplicate reads don't provide new information about the genome.
- Removing duplicates improves true coverage and reduces false positives.

Tool: Picard MarkDuplicates

- Flags or removes duplicate reads, reporting duplication metrics.
- **What to look for:** A high duplication rate be caused by over-amplification in library prep or technical bias.

Insert Size Distribution (Paired-End)

- Measures the fragment length between forward and reverse read pairs.
- Should align with library prep expectations (e.g., ~300 bp inserts).

Why do we care?

- Abnormal insert sizes could indicate contamination, structural variants, or library prep inconsistencies.
- Consistency in insert size distribution supports more accurate variant calling.

Tool: `samtools stats`

- Provides detailed alignment statistics, including insert size and base composition.
- **What to look for:** Do observed insert sizes match library prep?

Variant Filtering: Base Quality Score Recalibration (BQSR)

1. BaseRecalibrator:

- Uses known variant sites (e.g., dbSNP, 1,000 Genomes “gold standard”) to distinguish true mismatches from sequencer biases.
- Creates a recalibration table by comparing observed versus expected error rates across multiple covariates (e.g., read cycle, sequence context).

2. ApplyBQSR:

- Adjusts base quality scores in the BAM/CRAM using the recalibration table.
 - Output is a new file with more accurate base qualities.
-
- Overestimated quality scores can lead to false positive detection of variants.
 - Undercorrected reads can mask low-frequency or borderline variants. (= improves sensitivity)

Variant Filtering: Variant Quality Score Recalibration (VQSR)

1. VariantRecalibrator:

- Learns the distribution of annotation values for high-confidence “true” variant sites (training sets like HapMap, 1kGP, Omni).
 - More nuanced than simple “hard filters,” especially in large or complex datasets.-> **Maintains Sensitivity:**
- Constructs a Gaussian mixture model that differentiates likely true variants from false positives.

Even after accurate read alignment, some variants remain questionable due to noise or complex regions (e.g., homopolymers). VQSR uses a machine-learning model to filter out these “false variants.”

Variant Filtering: Variant Quality Score Recalibration (VQSR)

2. ApplyVQSR:

- Applies the model to the full variant set.
- Produces recalibrated scores or filters that retain high-confidence variants and remove suspicious calls.
 - Fewer false positives by penalizing variants with suspicious annotation profiles.->**Better specificity**

Variant Filtering: BQSR + VQSR

1. Perform **Read-Level Correction (BQSR)** after alignment, before variant calling.
 - Make sure base qualities fed into the variant caller accurately reflect real sequencing errors.
2. Perform **Variant-Level Refinement (VQSR)** after generating raw variant calls
 - Removes artifacts the caller may have flagged as variants.

Why do we need this?

- BQSR and VQSR together reduce both read-level and variant-level systematic errors. This is **essential for** clinical diagnostics, population genomics, and any application requiring high quality, reproducible variant sets.

Variant Calling

- The process of identifying genetic variants—differences from a reference genome—in sequencing data
- Typically focuses on SNPs (Single Nucleotide Polymorphisms) and indels (insertions/deletions), although some pipelines also address structural variants.

Germline variant callers assume that the variants are present in all cells and typically work with higher allele frequencies, making them well-suited for analyses where the genome is largely uniform.

Somatic variant callers are optimized to detect variants that may be present at low frequencies due to tumor heterogeneity or mosaicism, often necessitating paired analysis to distinguish somatic mutations from germline background.

Germline Variant Callers

Characteristic	Germline Variant Callers
Purpose	Identify inherited variants present in all cells; assume a stable, diploid (or polyploid) genome.
Allele Frequency	Expect high allele fractions: heterozygous (~50%) or homozygous (~100%).
Data Requirements	Typically run on single samples or cohorts; analyses are usually unpaired.
Statistical Models	Rely on models that assume a fixed ploidy and well-defined genotype priors.
Variant Filtering	Use population databases (e.g., dbSNP, 1kGP) and quality metrics to filter out sequencing artifacts.
Tools	GATK HaplotypeCaller, FreeBayes, DeepVariant, bcftools.
Performance	Generally optimized for detecting common, high-confidence variants with less stringent sensitivity requirements.

Somatic Variant Callers

Characteristic	Somatic Variant Callers
Purpose	Detect variants acquired in specific tissues (e.g., tumors); variants may be present in only a subset of cells.
Allele Frequency	Often exhibit low and variable allele frequencies due to tumor heterogeneity, subclonality, or mosaicism.
Data Requirements	Frequently require tumor-normal paired samples to distinguish somatic mutations from germline variants. Must account for mixed cell populations, variable purity, and complex copy number
Statistical Models	alterations. Incorporate filters for strand bias, low variant allele frequency, and tumor-specific
Variant Filtering	artifacts. GATK Mutect2, Strelka (somatic mode), LoFreq, Sentieon TNScope.
Tools	Require high sensitivity and specificity to detect low-frequency events while managing high background noise.
Performance Considerations	

Choosing Variant Caller by Algorithm

Approach / Type	Pros	Cons
Local Reassembly-Based (e.g., GATK HaplotypeCaller, Mutect2, FreeBayes, Strelka)	- Higher accuracy in complex regions (indels, multi-allelic sites) - Reduced false positives via local haplotype reconstruction - Strong community support and documentation	- Computationally heavy (higher CPU/RAM) - Longer runtime due to local assembly - More parameters to tune
Pileup-Based (e.g., bcftools, LoFreq)	- Faster and more lightweight - Simpler, straightforward implementation - Flexible (some specialize in low-frequency detection)	- May struggle with large indels or repetitive regions - Limited contextual insight (no local reassembly) - Can require careful threshold tuning in complex or high-coverage datasets

Choosing Variant Caller by Algorithm

Approach / Type	Pros	Cons
Deep Learning-Based (e.g., DeepVariant)	- High accuracy due to complex pattern recognition - Adaptable to multiple sequencing platforms (Illumina, PacBio, ONT) - Rapidly evolving with potential for further gains	- Resource-intensive (GPU/CPU for training/inference) - Dependent on quality and representativeness of training data - More complex setup (ML frameworks, dependencies)
Commercial Solutions (e.g., Sentieon)	- Often faster with lower memory usage vs. open-source references - Paid support and regular updates (useful in clinical/regulatory settings) - Deterministic, reproducible runs	- License fees and usage restrictions - Closed-source (limited transparency/customization) - Community support may be smaller or tied to vendor

Germline Variant Calling

Germline \ Approach	Local Reassembly-Based	Pileup-Based	Deep Learning	Commercial Solutions
Tools	- GATK HaplotypeCaller - FreeBayes	- bcftools	- DeepVariant	- Sentieon DNAscope - Sentieon Haplotyper
Data Type	- Mostly short-read Illumina - FreeBayes can handle some long-read with tuning - Single-sample or cohort	- Primarily short-read Illumina - Single or multi-sample	- Short or long reads (separate models) - Typically single-sample	- Mostly short-read Illumina - Single or multi-sample
Resources	- Moderate/high CPU & RAM (HaplotypeCaller tends to be heavier)	- Lightweight, fast execution	- Requires GPU or multi-CPU for best performance	- Optimized for speed & lower RAM than GATK
License	- GATK: free for academic (commercial restrictions) - FreeBayes: open-source (MIT)	- bcftools: open-source (MIT/BSD)	- DeepVariant: open-source (Apache 2.0)	- Sentieon: commercial (paid license)

Somatic Variant Calling

Somatic \ Approach	Local Reassembly-Based	Pileup-Based	Commercial Solutions
Tools	- GATK Mutect2 - Strelka (somatic mode)	- bcftools (less common) - LoFreq (also somatic)	Sentieon TNScope
Data Type	- Short-read Illumina - Typically tumor-normal pair (Strelka can do tumor-only in some modes)	- Short-read Illumina - LoFreq also good for low-freq somatic	- Short-read Illumina, tumor-normal
Resources	- Mutect2: moderate/high CPU - Strelka: relatively fast	- bcftools: lightweight - LoFreq: moderate resource use	- Faster than Mutect2, lower RAM usage
License	- GATK: free for academic use (commercial restrictions may apply) - Strelka: open source (BSD)	- bcftools: open source - LoFreq: open source (BSD)	- Sentieon: commercial (paid license)

Note: LoFreq is often used for low-frequency somatic variants, but it can also be applied in standard somatic calling.

Low Frequency Variant Calling

Low-Frequency \ Approach	Local Reassembly-Based	Pileup-Based
Tools	- GATK Mutect2 (subclonal detection) - Strelka (supports low VAF)	- LoFreq (optimized for rare alleles)
Data Type	- Short-read Illumina - High coverage for subclonal detection - Similar to general somatic calling	- Short-read Illumina - Viral or subclonal
Resources	- Higher coverage requires more CPU/Time	- Moderate resource usage; scales with coverage
License	- GATK: free for academic - Strelka: open source (BSD)	- LoFreq: open source (BSD)